

JavaScript Notes — Day 11

Chapter 11: Object-Oriented Programming (OOPs)

Objects with Methods — the "this" Keyword

An object can hold functions as properties (called methods). Inside a method, the "this" keyword refers to the object the method was called on, so it can access that object's own properties.

```
const student = {
  fullname: "chiru",
  marks: 92,
  printMarks: function () {
    console.log("marks =", this.marks); // this = student
  }
};

student.printMarks(); // marks = 92
```

Prototypes

Every object can be linked to another object, called its prototype, using `__proto__`. The object then gets access to the prototype's properties and methods — this is how JavaScript shares behavior between objects before classes existed.

```
const emp = {
  calcTax() {
    console.log("tax rate is 11%");
  }
};

const chiru = {
  salary: 50000,
};

chiru.__proto__ = emp; // chiru now has access to calcTax()
chiru.calcTax(); // "tax rate is 11%"
```

Note: `__proto__` works but is considered old-style; `Object.setPrototypeOf()` / `Object.create()` are the modern equivalents. Classes (below) handle this automatically.

Classes

A class is a blueprint for creating objects that share the same structure and methods. Use the "new" keyword to create (instantiate) an object from a class.

```
class ToyotaCar {
  start() {
    console.log("start");
  }
}
```

```

    }

    stop() {
        console.log("stop");
    }

    setBrand(brand) {
        this.brandName = brand;
        console.log("this is:", this.brandName);
    }
}

// creating an object from the class
let fortuner = new ToyotaCar();
fortuner.setBrand("tesla"); // this is: tesla

```

Inheritance (extends)

A child class can reuse the properties and methods of a parent class using the extends keyword. This avoids rewriting the same code in multiple classes.

```

class Parent {
    constructor() {
        this.info = "parent of the children";
    }
    hello() {
        console.log("hello");
    }
}

class Child extends Parent {}

let c = new Child();
c.hello(); // "hello" – inherited from Parent

```

The super Keyword

Inside a child class, `super()` calls the parent class's constructor, and `super.methodName()` calls a method from the parent class. `super()` must be called before using "this" in the child's constructor.

```

class Person {
    constructor() {
        this.species = "homo sapiens";
    }
    eat() {
        console.log("eat");
    }
}

class Eng extends Person {

```

```

    constructor(branch) {
        super(); // runs Person's constructor first
        this.branch = branch;
    }

    work() {
        super.eat(); // call the parent's method
        console.log("do some projects");
    }
}

let me = new Eng("computer science");
me.work(); // "eat" then "do some projects"

```

Practice Problem — User and Admin Classes

Qs: Create a class User with properties name and email, and a method viewData() to view website data. Then create a class Admin that inherits from User and adds a method editData() to edit website data.

```

let DATA = "secret information";

class User {
    constructor(name, email) {
        this.name = name;
        this.email = email;
    }

    viewData() {
        console.log("data =", DATA);
    }
}

class Admin extends User {
    constructor(name, email) {
        super(name, email);
    }

    editData() {
        DATA = "some new value";
    }
}

let s1 = new User("chiru", "cp10547@gmail.com");
let admin = new Admin("me", "goat@edu.atria");

admin.viewData(); // inherited from User
admin.editData(); // Admin's own method

```

Note: DATA here is a variable shared across all instances (module-level), not a per-object property — every User/Admin object reads and writes the same DATA.

Error Handling — try...catch

try...catch lets you run code that might fail without crashing the whole program. If an error occurs inside try, execution jumps straight to catch, skipping the rest of the try block; code after the try...catch block continues running normally.

```
try {  
  // code that might throw an error  
} catch (err) { // err is the error object  
  // handle the error here  
}
```

Example

```
let a = 5;  
let b = 10;  
console.log("a =", a);  
console.log("b =", b);  
console.log("a+b =", a + b); // 15  
  
try {  
  console.log("a+b=", a + c); // c is not defined → throws an error  
} catch (err) {  
  console.log(err); // ReferenceError: c is not defined  
}  
  
// execution continues normally after the catch block  
console.log("a+b =", a + b); // 15
```

Note: a + c fails because c was never declared — that's exactly the kind of runtime error try...catch is meant to catch, instead of the whole script stopping.

Quick Summary

Concept	What it does
this	Refers to the object a method is called on
__proto__	Links an object to another object to share its methods
class	Blueprint for creating objects with shared structure/methods
new	Creates an instance (object) from a class
extends	Lets a child class inherit from a parent class
constructor()	Special method that runs when an object is created with new
super()	Calls the parent class's constructor
super.method()	Calls a specific method from the parent class

try...catch	Runs risky code and handles any error without crashing
-------------	--